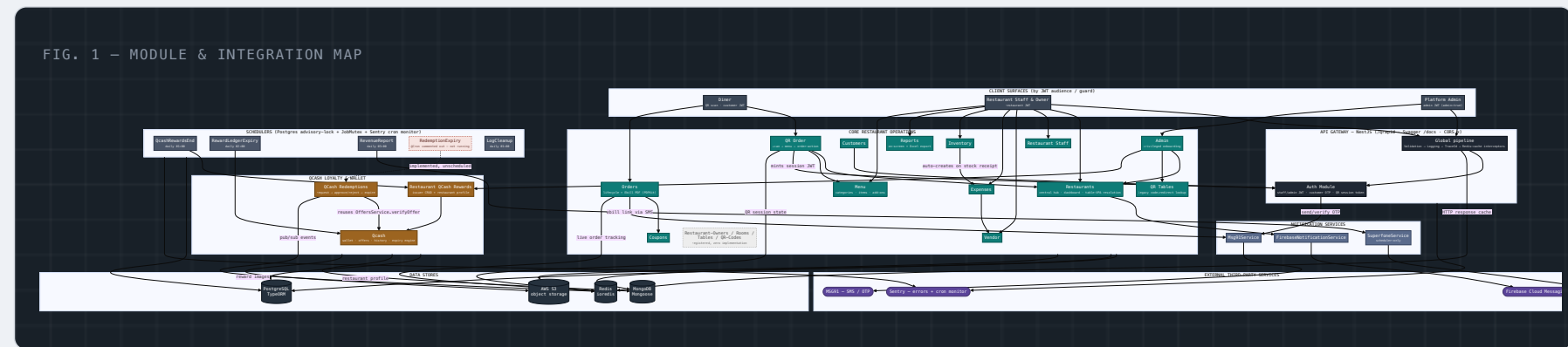
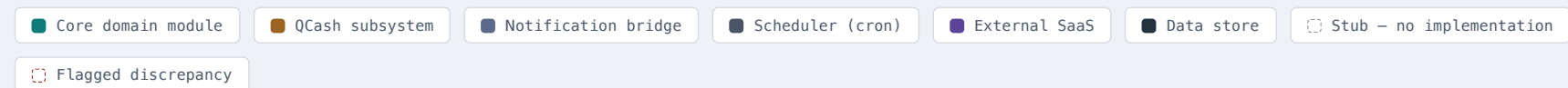


QRapid Backend – System Architecture

A ground-truth map of the NestJS monolith behind QRapid's QR-ordering, restaurant-operations, and QCash loyalty platform. Every module, data store, and third-party integration below was confirmed by reading `app-dash-board-backend-2-main — controllers, services, module wiring, and package.json`. No PRD exists in this repository or its immediate workspace, so nothing here is inferred beyond what the code implements.

Framework: NestJS / TypeScript Global prefix: /qrapid API docs: /docs (Swagger) Reviewed: 2026-07-19 Modules registered in AppModule: 20



FINDINGS

Notes on what the code actually does

"sql-config" is Postgres, not MySQL

`src/infrastructure/sql-config/mysql.module.ts` hardcodes `type: 'postgres'` and `DB_TYPE=postgres` is set in `env/local.env`. The `mysql2` package in `package.json` is unused — file/folder naming is a leftover, not a second database.

Four modules are empty stubs

`RestaurantOwners`, `RestaurantQrCodes`, `RestaurantTables`, and `RestaurantRooms` register a controller and service with no methods. Real table/room/QR-code logic lives inside `RestaurantsService` and `AdminService` instead.

Two unrelated “QR table” concepts

The `QrTable` entity (admin-managed code→redirect-link lookup) has no foreign key to `RestaurantTable.qrpidVPA` — the field the live QR-order menu-scan flow actually resolves against. They appear to be a legacy mechanism and the current one, coexisting.

SuperfoneService lives in the wrong module

Despite sitting in `src/notification/`, `SuperfoneService` is only provided by `SchedulersModule`, used solely by the daily revenue-report job. Its `sendOTPMMessage` and `sendEbillMessage` methods exist but are never called — OTP and e-bill notices both go through MSG91 instead.

Redemption-expiry sweep isn't scheduled

In `redemption-expiry.scheduler.ts` the `@Cron` decorator is commented out. The expiry logic is fully implemented but does not currently run automatically.

E-bill README is stale

`EBUILD_README.md` describes an HTML→Puppeteer pipeline that no longer exists. The live `ebill-pdf.service.ts` generates PDFs natively with `pdfkit` — no headless-Chrome dependency appears anywhere in `package.json`.

Two AWS S3 SDKs coexist

`S3StorageService` (AWS SDK v3, current — reports, e-bills, menu images) and `S3StoragePublicService` (legacy SDK v2) both write to S3, drawn here as one data store.

No PRD in scope

No product-requirements document exists in this repository or its immediate workspace — only a pitch deck and an operations-flow doc were found nearby, and neither is a PRD. This diagram is built from source code alone, as instructed.

Module	Route Prefix	Purpose	Primary Stores
Restaurants	v2/restaurants	Central restaurant entity — dashboard metrics, GST/partner details, printer config, FCM token storage, table-VPA resolution	Postgres
Restaurant Staff	v2/restaurant-staff	Staff account CRUD under a restaurant	Postgres
Menu	v1/menu	Categories, items, variations, add-ons	Mongo S3
QR Order	v2/qr-order	QR-scan → menu fetch → session token → order-action / bill lookup	Postgres Mongo Redis
Orders	v2/orders	Order lifecycle, live tracking, payment, e-bill PDF + SMS	Postgres Redis S3
Customers	v2/customers	Profile, order history, referral code, account deletion	Postgres
Coupons	v1/coupons	Create / verify / inactivate discount coupons	Postgres
Vendor	v1/vendors	Supplier / vendor records	Postgres
Expenses	v1/expenses	Operating expense tracking, bulk settle, reports	Postgres
Inventory	v1/inventory	Stock/ledger tracking, consumption & wastage reports	Postgres
Reports	v2/reports	On-screen reports + async Excel export to S3	Postgres Mongo S3
Admin	v2/admin, v2/admin/qcash	Restaurant onboarding, activation, QR-table mapping, QCash-restaurant onboarding — admin: true JWT only	Postgres
QR Tables	v1/qr-tables	Legacy code → redirect-link lookup, admin-managed	Postgres
Auth	v2/login, v1/otp	Staff/admin JWT (email+password) and customer OTP (via MSG91) — two independent flows	Postgres MSG91
Qcash	v1/qcash	Customer wallet balance/ledger, offer browsing, reward-expiry engine	Postgres
Restaurant QCash Rewards	v1/restaurant-qcash-rewards	Reward-campaign CRUD (issuer side), restaurant profile, image upload	Postgres Mongo S3
QCash Redemptions	v1/qcash-redemptions	Request → approve/reject → expire state machine on the reward ledger	Postgres Redis
Notification	v2/notification	Push (Firebase), SMS/OTP (MSG91), WhatsApp (Superfone, scheduler-only)	Firebase MSG91 Superfone
Schedulers	—	5 cron jobs: QCash reward end, reward-ledger expiry, daily revenue report, redemption expiry (unscheduled), log cleanup	Postgres
Restaurant Owners / Rooms / Tables / QR-Codes	—	Registered NestJS modules with no implemented logic	stub